План управления конфигурацией ПО

Проект: Система автоматизации управления жизненным циклом релиза ПО

Версия 1 от 17.10.2026

Авторы: Заранкович Д. Н. (капитан), Брылкина К. К., Поминова А. А., Пономаренко А. А., Карцева П. А., Лазарев Я. О., Деньков Д. С., Чернов А. В.

Курс «Технология программирования кибернетических систем: управление проектами» (ТПКС) · преподаватель Козырев В. П. · НИЯУ МИФИ, 2026

Содержание

1 Введение **2**
- 1.1 Цель . . . 2
- 1.2 Область применения . . . 2
- 1.3 Ссылки . . . 2
- 1.4 Термины и сокращения . . . 2

2 Среда управления конфигурацией ПО **3**
- 2.1 Инструменты УК ПО . . . 3
- 2.2 Организация и ответственность . . . 4
- 2.3 Единицы конфигурации и категории контроля . . . 4
- 2.4 Средства трассировки . . . 5
- 2.5 Связь с другими процессами ЖЦ ПО . . . 5

3 Мероприятия процесса УК ПО **6**
- 3.1 Идентификация конфигурации . . . 6
- 3.2 Базовые версии и трассируемость . . . 6
  - 3.2.1 Создание базовой версии . . . 6
  - 3.2.2 Контроль библиотек . . . 6
  - 3.2.3 Трассируемость версий . . . 6
- 3.3 Регистрация и отслеживание проблем . . . 6
- 3.4 Управление изменениями: СП и ЗИ . . . 7
- 3.5 Рассмотрение изменений . . . 7
- 3.6 Интеграция изменений . . . 7
- 3.7 Хранение, воспроизведение и выпуск . . . 7
- 3.8 Контроль загрузки ПО . . . 8
- 3.9 Контроль среды жизненного цикла ПО . . . 8
- 3.10 Учёт состояния конфигурации . . . 8

4 Документы управления конфигурацией **8**

5 Критерии перехода **8**

Лист регистрации изменений **9**

1 Введение

1.1 Цель

План задаёт порядок управления конфигурацией программного обеспечения проекта. УК ПО должно сохранять целостность данных жизненного цикла: участники должны работать с известными версиями, изменения должны иметь основание и результат, а утверждённую конфигурацию должно быть возможно воспроизвести.

1.2 Область применения

План действует для документов, требований, исходного кода, сценариев проверки, результатов сборки и записей об их изменениях. Он применяется от начала подготовки изменения до выпуска и загрузки утверждённой конфигурации. Объектный код, полученный компилятором, не редактируется и не является самостоятельной ЕК, но его версия и контрольная сумма фиксируются в протоколе интеграции.

1.3 Ссылки

- DO-178C и КТ-178C, раздел 7 «Управление конфигурацией ПО»;
- системные требования проекта, версия 3;
- слайды В. П. Козырева «УК ПО», страницы 49–94;
- стандарт на разработку требований к ПО проекта (после утверждения).

1.4 Термины и сокращения

Базовая версия — утверждённая версия единицы конфигурации, служащая основой последующей работы. Промежуточные версии базовыми не считаются (источник: слайды 32, 75–76).

ЕК — единица конфигурации: данные жизненного цикла ПО, рассматриваемые при управлении конфигурацией как одно целое и определяемые уникальным именем и версией (источник: слайды 52–53).

ГУИ — группа управления изменениями: постоянная часть совета по управлению изменениями (источник: слайд 71).

ЗИ — запрос на изменение: описание планируемого изменения одной ЕК, порождённое утверждённым сообщением о проблеме (источник: слайды 73–74).

КК1 — категория контроля данных, для которой изменение после создания базовой версии выполняется через запрос на изменение (источник: конспект, К:16, 61).

КК2 — категория контроля данных с менее строгим набором мероприятий, чем КК1; конкретный порядок задаёт План управления конфигурацией ПО (источник: конспект, К:16, 61, 639; наша формулировка).

СКУ — система конфигурационного управления: средства учёта версий, изменений и совместной работы с данными проекта (источник: слайд 85).

СП — сообщение о проблеме: запись об одном несоответствии результатов исходным данным, данных стандартам или процессов планам (источник: слайды 64–66).

УК ПО — управление конфигурацией ПО: учёт состава, версий и изменений данных жизненного цикла для сохранения целостности ПО (источник: слайды 49, 89).

В этом плане используются также термины GitHub, issue, pull request, ветка, тег, релиз

и артефакт сборки. Их значения определены контекстом GitHub и правилами данного плана. Issue используется для записи СП или ЗИ, но тип записи всегда указывается меткой.

2 Среда управления конфигурацией ПО

2.1 Инструменты УК ПО

Единственным разрешённым источником данных проекта служит репозиторий команды на GitHub. В нём хранятся документы в каталоге `docs`, готовые PDF в `pdf`, исходный код, сценарии проверки, файлы сборки и записи процесса. Папка `materials` содержит исходные учебные материалы и не изменяется в рамках разработки.

Таблица 1 – Инструменты и назначение

Инструмент	**Назначение**
Git	Хранение версий файлов, веток, коммитов и тегов.
GitHub Issues	Регистрация СП и ЗИ, их статусы, обсуждение и трассировка.
GitHub Pull requests	Рассмотрение изменений, обсуждение строк и разрешение слияния.
GitHub Actions	Автоматическая сборка и запуск проверок после создания pull request и после слияния.
Хранилище артефактов	Хранение утверждённых PDF, протоколов сборки и выпусков.

Доступ к репозиторию предоставляется только участникам команды. Право чтения нужно для просмотра конфигурации и проверки результатов, право записи — для создания веток, коммитов и pull request. Слияние в `main` разрешено только проверяющему изменения или ответственному за выпуск после выполнения критериев перехода из раздела 5.

2.2 Организация и ответственность

Таблица 2 – Роли процесса УК ПО

Роль	**Ответственность**
Совет по управлению изменениями	Коллегиально решает по СП, нужны ли изменения и какие ЕК затрагиваются.
ГУИ	Постоянная часть совета; организует рассмотрение и контролирует состояние СП и ЗИ.
Руководитель проекта	Назначает автора ЗИ и проверяющего, разрешает спорные организационные вопросы.
Автор изменений	Вносит изменение только в пределах назначенного ЗИ, обновляет трассировку и описание результата.
Проверяющий изменения	Проверяет соответствие изменения ЗИ и входным данным, анализирует результаты CI и принимает решение по pull request.
Ответственный за выпуск	Формирует выпуск из утверждённых ЕК, публикует его и контролирует загрузку.

Автор изменения не может быть единственным проверяющим этого изменения. Состав ГУИ и правило назначения МИ, указанного на схеме жизненного цикла ЗИ в слайдах, нужно согласовать с командой: [уточнить: кто в проекте исполняет роль МИ].

2.3 Единицы конфигурации и категории контроля

ЕК однозначно определяется парой «уникальное имя, версия». Составная ЕК имеет собственную версию и содержит версии входящих в неё ЕК. Версия ЕК увеличивается только по утверждённому ЗИ.

Таблица 3 – Единицы конфигурации

ЕК	**Категория**	**Идентификация и хранение**
Системные требования и СПТ	КК1	Файл с именем документа и версией; исходный текст и PDF в Git.
Планы и стандарты	КК1	Файл документа с номером версии; исходный текст и PDF в Git.
Исходный код и сценарии проверки	КК1	Путь файла, коммит и версия репозитория.
СП и ЗИ	КК1	Issue с уникальным номером, типом и списком затронутых ЕК.
Pull request	КК2	Номер репозитория и PR, связанные с ЗИ и коммитами.
Протоколы ФИ, сборки и загрузки	КК2	Файл протокола с датой, версиями входов и результатом.
Конфигурация CI и зависимости	КК1	Файл конфигурации, коммит и зафиксированные версии зависимостей.
Выпуск ПО и его манифест	КК2	Тег релиза, список ЕК и контрольные суммы артефактов.

КК1 означает, что после создания базовой версии изменение допускается только по ЗИ. Для КК2 достаточно фиксировать версию, автора, дату и результат операции; порядок повторной проверки задаётся видом протокола. Объектный код не редактируется и поэтому не получает отдельную категорию контроля.

2.4 Средства трассировки

Каждая запись СП перечисляет ЕК, в которых обнаружена проблема. Утверждённое СП связано с одним или несколькими ЗИ, а каждый ЗИ — с одной изменяемой ЕК, веткой, pull request и итоговым коммитом. Манифест выпуска связывает версии всех включённых ЕК с артефактами и протоколом загрузки. Связь между версиями одной ЕК проходит через ЗИ; промежуточные коммиты не считаются базовыми версиями.

2.5 Связь с другими процессами ЖЦ ПО

Разработка требований, проектирование и кодирование получают ЗИ как основание изменения и передают результаты в УК ПО для проверки и фиксации. Интеграция создаёт исполняемый объектный код, протокол сборки и протокол загрузки. Верификация и гарантия качества регистрируют найденные несоответствия через СП. Планирование определяет инструменты и правила, которые далее контролируются этим планом.

3 Мероприятия процесса УК ПО

3.1 Идентификация конфигурации

Перед началом работы ГУИ определяет состав данных жизненного цикла и их связи. Для каждой ЕК задаются уникальное имя, расположение, категория контроля и правило версии. Идентификатор документа состоит из имени файла и номера версии; исходный код идентифицируется путём и коммитом; issue — номером и меткой; pull request — номером PR; выпуск — тегом.

Ветка `main` содержит только проверенные изменения. Рабочая ветка создаётся для одного ЗИ и именуется `zi-<номер>-<краткое-описание>`. Коммиты должны содержать номер ЗИ, например `ZI#17: update SCM plan`. Удаление коммитов и истории после слияния запрещено.

3.2 Базовые версии и трассируемость

3.2.1 Создание базовой версии

Рабочие версии могут существовать в ветке ЗИ, но базовой становится только версия, которая прошла проверку, утверждена и передана в СКУ. Для документов базовая версия фиксируется коммитом в `main`, а для выпуска ПО дополнительно тегом вида `v<major>.<minor>.<patch>`. Формулировка «самый новый коммит `main`» означает только последний коммит, вошедший в `main` по утверждённому ЗИ; она не делает любой коммит автоматически базовым.

3.2.2 Контроль библиотек

Файлы зависимостей входят в КК1. В них фиксируются версии библиотек и контрольные суммы или lock-файлы. CI устанавливает зависимости только из разрешённого источника и сохраняет протокол установки. Обновление зависимости выполняется отдельным ЗИ с анализом влияния.

3.2.3 Трассируемость версий

Перед слиянием проверяющий убеждается, что PR связан с ЗИ, ЗИ связано с утверждённым СП, а список изменённых файлов совпадает со списком ЕК в ЗИ. После слияния в ЗИ записываются хэш коммита, версии документов и ссылка на протокол CI.

3.3 Регистрация и отслеживание проблем

СП регистрирует одно несоответствие между входами и выходами процессов, данными жизненного цикла и стандартами либо процессами и планами. Зарегистрировать СП может любой участник проекта.

В СП указываются уникальный номер, краткое описание проблемы, шаги воспроизведения, фактический и ожидаемый результат, список затронутых ЕК и их версий. СП описывает проблему, а не предполагаемое решение. Для записи используется метка `type::problem`.

Жизненный цикл СП:

1. «Создано» — запись ещё не проверена;
2. «Зарегистрировано» — ГУИ проверила обязательные поля;
3. «Рассматривается» — совет оценивает проблему и влияние;
4. «В работе» — по СП выполняются утверждённые изменения;

5. «Завершено» — изменения внесены, проверены и связаны с результатом;
6. «Отклонено» — запись создана ошибочно или изменений не требуется.

3.4 Управление изменениями: СП и ЗИ

Совет по управлению изменениями рассматривает зарегистрированное СП. Он может отклонить СП, если проблема подтверждена ошибочно или изменение не требуется, либо утвердить его и перечислить затрагиваемые ЕК. Из одного СП порождается несколько ЗИ, по одному на каждую изменяемую ЕК.

ЗИ описывает планируемое изменение, его основание, исходную версию ЕК, критерий готовности и способ проверки. Для него используется метка `type::change`. Жизненный цикл ЗИ:

1. «Создан» — ЗИ создан из утверждённого СП;
2. «Передан в работу» — руководитель проекта назначил автора;
3. «В работе» — автор меняет ЕК в рабочей ветке;
4. «Проверка» — изменение готово, проверяющий выполняет ревью и проверки;
5. «Завершён» — изменение утверждено, слито и зафиксировано;
6. «Отклонён» — изменение ошибочно или больше не требуется.

3.5 Рассмотрение изменений

Автор создаёт ветку ЗИ от актуального `main`, вносит только относящиеся к ЗИ коммиты и открывает pull request. В описание PR он добавляет ссылки на СП и ЗИ, перечень ЕК, описание проверки и ожидаемый результат. CI запускает сборку, проверку формата, тесты и проверку ссылок.

Проверяющий сравнивает изменение с ЗИ, проверяет трассируемость, результаты CI и отсутствие открытых замечаний. Если есть несоответствие, PR возвращается автору, а ЗИ остаётся в состоянии «В работе». После устранения замечаний проверяющий повторяет проверку. Только после одобрения PR ответственный за слияние выполняет merge в `main`; удалять PR и историю обсуждения запрещено.

3.6 Интеграция изменений

После слияния CI выполняет интеграционную сборку из коммита `main`. Входами являются исходный код, параметры сборки, описание проекта, компонент параметрических данных (если он есть), План разработки ПО и ЗИ. Выходами являются исполняемый объектный код, протокол сборки, командные файлы сборки, протокол загрузки и сообщения о проблемах.

Артефакты и протоколы получают идентификаторы, связываются с коммитом `main` и сохраняются в хранилище. Если сборка или загрузка завершилась ошибкой, CI создаёт СП с указанием версии входов. Объектный код не изменяется вручную и не используется как источник для следующей сборки.

3.7 Хранение, воспроизведение и выпуск

GitHub защищает `main` от прямой записи и хранит историю коммитов, issue и PR. Резервная копия репозитория и хранилища артефактов выполняется регулярно; ответственный за УК проверяет возможность чтения копии. Воспроизведение конфигурации выполняется по тегу выпуска, манифесту ЕК, коммиту `main` и протоколу сборки.

Ответственный за выпуск проверяет, что все ЕК выпуска имеют утверждённые версии, PR слиты, СП и ЗИ закрыты, а протокол интеграции не содержит открытых несоответствий. Затем он создаёт тег, формирует манифест и публикует выпуск в хранилище. Непроверенные или изменённые вручную файлы в выпуск не включаются.

3.8 Контроль загрузки ПО

Перед загрузкой ответственный за выпуск сверяет тег, манифест и контрольные суммы артефактов. Загрузку выполняют только из опубликованного выпуска. После загрузки фиксируются целевая среда, версия аппаратуры, время, исполнитель, контрольная сумма и результат проверки. Несовпадение контрольной суммы или несовместимость с аппаратурой блокирует загрузку и регистрируется СП.

3.9 Контроль среды жизненного цикла ПО

Версии Git, среды сборки, инструментов проверки, зависимостей и раннеров CI перечисляются в файле конфигурации проекта. Этот файл является ЕК КК1. Изменение инструмента или его версии выполняется по ЗИ с проверкой воспроизводимости сборки. Доступы к GitHub и хранилищу выдаются по роли и отзываются при уходе участника из проекта.

3.10 Учёт состояния конфигурации

Состояние конфигурации отражается в GitHub и в манифесте выпуска. Для каждой ЕК доступны её версия, категория, владелец изменения, связанное СП или ЗИ, состояние проверки и базовая версия. ГУИ перед выпуском сверяет таблицу состояний со списком файлов, PR, протоколами и тегом.

4 Документы управления конфигурацией

К документам УК ПО относятся настоящий план, шаблоны СП и ЗИ, правила веток и коммитов, матрица ЕК, манифесты выпусков, протоколы CI, сборки и загрузки, резервные копии и отчёты о состоянии конфигурации. У каждого документа есть версия и запись в листе изменений. Утверждённые документы хранятся в `docs` и PDF в `pdf`; рабочие черновики не являются базовыми версиями.

5 Критерии перехода

Таблица 4 – Критерии перехода

Переход	**Условие**
СП создано → зарегистрировано	Заполнены идентификатор, одна проблема, затронутые ЕК и их версии.
СП рассматривается → утверждено или отклонено	Совет зафиксировал решение и список ЕК.
ЗИ создано → передано в работу	Назначены автор и проверяющий, указаны критерии готовности.
ЗИ в работе → проверка	PR открыт, CI завершён успешно, трассировка заполнена.
ЗИ проверка → завершён	Проверяющий одобрил PR, изменение слито в `main`, протокол сохранён.
Конфигурация → выпуск	Все ЕК утверждены, интеграция успешна, манифест и контрольные суммы сформированы.
Выпуск → загрузка	Ответственный сверил тег, манифест, артефакты и совместимость целевой среды.

Лист регистрации изменений

Версия	**Дата**	**Что изменено**	**Кто**	**Основание**
1	17.10.2026	Первая версия плана управления конфигурацией ПО	Лазарев Я. О.	Задание к паре 17.10.2026